

Ez Meeting

Video Conferencing with AI-Powered Summaries & Attention Detection

Project Scope, Flow & Tools Documentation | Version 2.0

1. Project Overview

Ez Meeting is a real-time video conferencing web application with AI-powered meeting summaries, document collaboration, voice captions (English), and participant attention detection using face/eye tracking. Built as a progressive web app (PWA) for instant access on any device.

Instant Meetings
Create or join with a 6-character code or QR scan. No downloads required.

AI Meeting Summary
Automatic post-meeting summary with topics, action items, and key decisions via DeepSeek.

Attention Detection
Real-time face/eye tracking with per-participant active percentage (host-only view).

Document Collaboration
Upload, analyze, and ask AI questions about documents during the meeting.

Voice Captions
Real-time English speech-to-text. Always-on transcript for AI summary.

Meeting History
All participants can view past meetings, summaries, and export reports.

2. Tech Stack & Tools

Layer	Technology	Purpose
Frontend	Vanilla JavaScript, HTML5, CSS3	Single-page app, no framework dependency
	WebRTC + simple-peer	Peer-to-peer video/audio streaming
	Socket.io (client)	Real-time signaling, chat, captions
	MediaPipe FaceLandmarker	Face/eye detection for attention scoring

	Browser SpeechRecognition API	English voice-to-text captions & transcript
	Lucide Icons	UI iconography
	PWA (Service Worker)	Installable on mobile
Backend	Node.js (v18+)	Server runtime
	Express.js	HTTP server & REST API
	Socket.io (server)	WebSocket signaling & events
	better-auth + JWT	User authentication & sessions
	Multer	File upload handling
	QRCode	Meeting QR code generation
AI	DeepSeek API	Meeting summaries, document analysis, Q&A (all AI functions)
Database	SQLite (better-sqlite3)	User accounts & sessions (local)
	Supabase PostgreSQL	Meeting history persistence (cloud)
Document	pdf-parse, mammoth, officeparser	PDF, Word, Excel, PowerPoint extraction
Deploy	Railway + Docker	Cloud deployment with auto-deploy on push

3. Application Flow

3.1 Login Flow

1. User opens app → Login page



2. Enter email + display name → Server issues JWT token



3. Token stored in browser localStorage → Redirected to Lobby

3.2 Meeting Flow

Host: Click "Start Meeting" → Server creates room with 6-char code → Share code/QR/link



Participant: Enter code or scan QR → Join room → WebRTC connections established with all peers

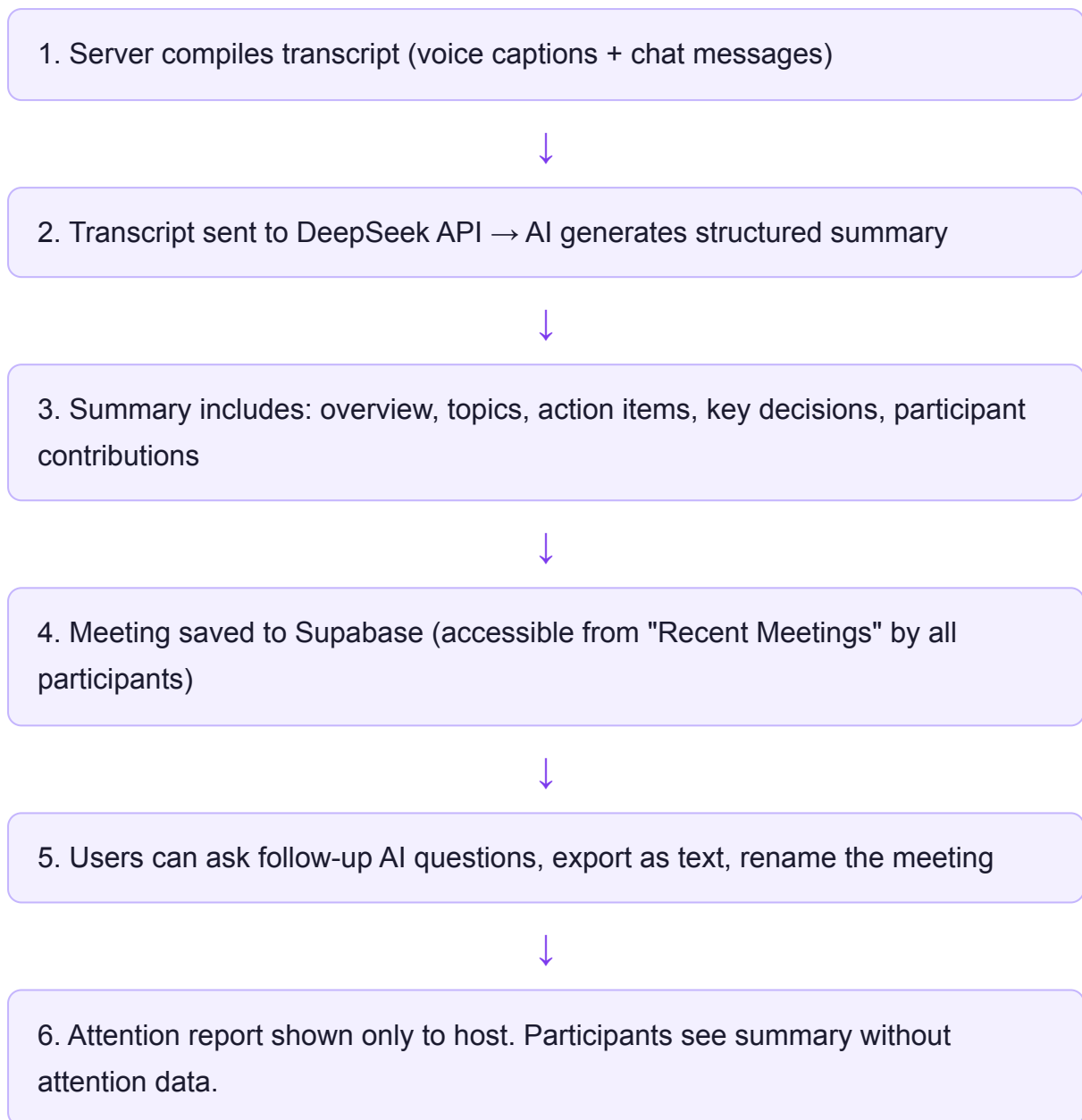


During Meeting: Video/audio streaming, chat, screen share, document upload, voice captions, attention detection all run simultaneously

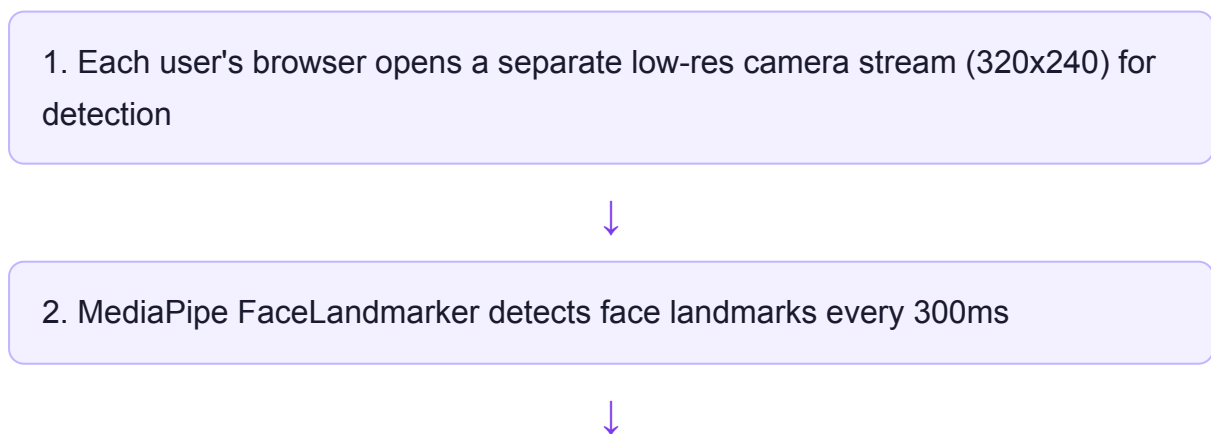


End Meeting: Host clicks "End for all" OR last person leaves

3.3 Post-Meeting Summary Flow



3.4 Attention Detection Flow



3. Per-frame scoring: Eye score (open=1, closed=0) $\times$ 0.6 + Head score (forward=1, slight turn=0.5, away=0) $\times$ 0.4



4. 3-second sliding window average $\rightarrow$ Status: Active (>70%), Semi-active (40-70%), Not active (<40%)



5. Score emitted to server every 1s $\rightarrow$ Server relays to host only $\rightarrow$ Host sees % on each participant's tile and overall meter

3.5 Document Q&A Flow

1. User uploads document (PDF, Word, Excel, PowerPoint, text, code, etc.)



2. Server extracts text $\rightarrow$ DeepSeek generates overview, key points, study questions, objectives



3. Insights shown in sidebar $\rightarrow$ User can ask questions $\rightarrow$ DeepSeek answers using document context (RAG)

4. Feature Details

4.1 Core Meeting Features

Feature	Description	Type
Video/Audio Calls	WebRTC peer-to-peer mesh. Each participant connects directly to all others.	Real-time
Text Chat	Socket.io messages with timestamps and sender names. Persisted in transcript.	Real-time
Screen Sharing	Host shares directly. Guests request permission. Shared screen shown in center featured tile.	Real-time
Camera Flip	Switch between front and back camera on mobile devices.	
Meeting Share	QR code, copy link, and room code display for inviting participants.	
Participants View	Full-screen overlay showing all participants with live video tiles. Host sees attention %.	
Spotlight View	Click any participant to see them in a large full-screen overlay. Close with X.	

4.2 AI Features (All DeepSeek)

Feature	Description	API
Meeting Summary	Auto-generated overview, topics, assignments, key decisions after meeting ends	DeepSeek
Meeting Q&A	Ask follow-up questions about any past or current meeting	DeepSeek
Document Analysis	Overview, key points, deep dive, study questions, objectives from uploaded files	DeepSeek
Document Q&A	Ask questions about uploaded documents (RAG-style)	DeepSeek

4.3 Attention Detection

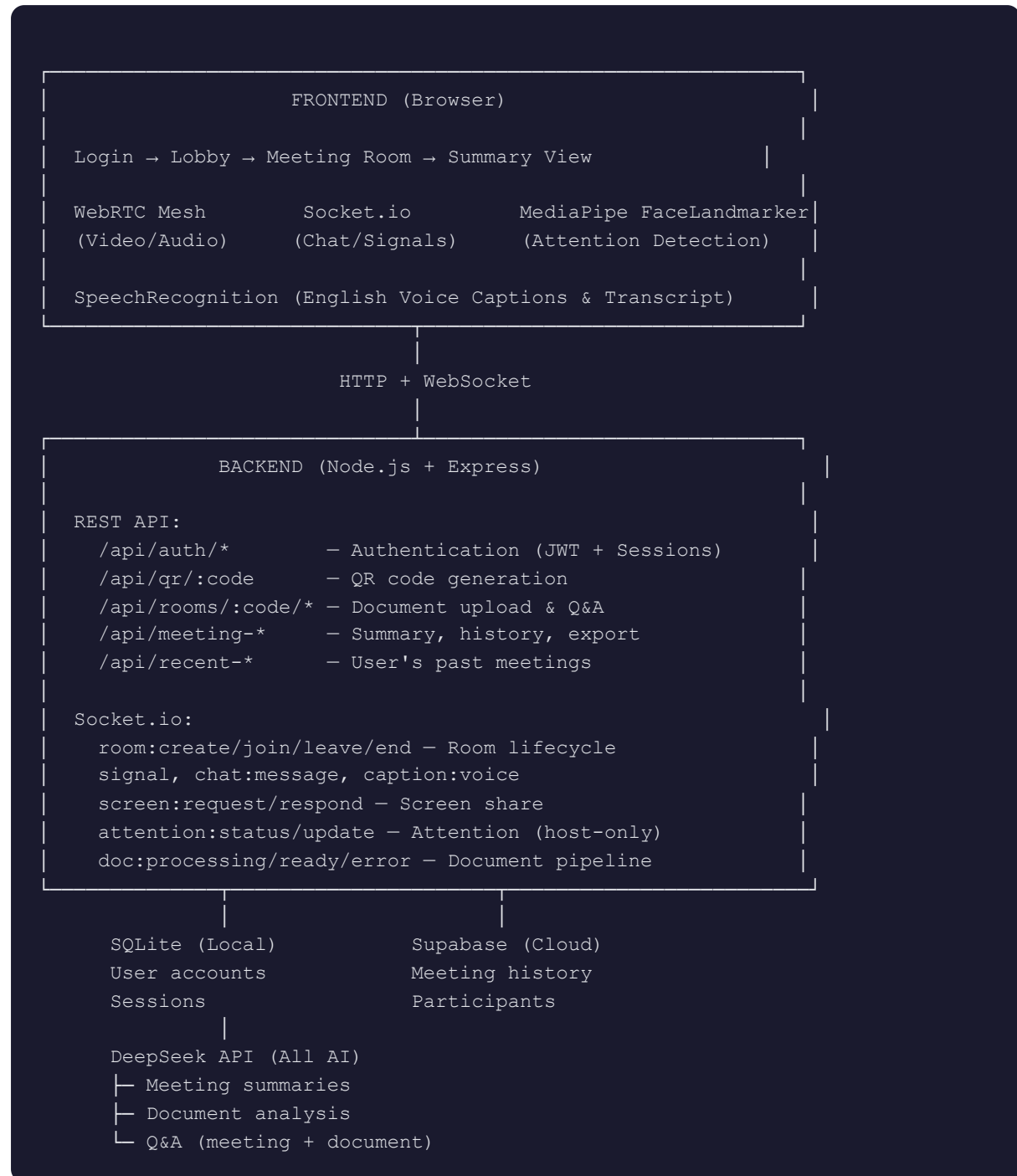
Component	Detail
Detection Engine	MediaPipe FaceLandmarker (WASM + GPU), separate 320x240 camera stream

Detection Rate	~3 fps (300ms interval)
Eye Scoring	Eye Aspect Ratio (EAR): open=1, closed=0. Blinks <350ms ignored.
Head Scoring	Forward=1, slight turn=0.5, looking away/down=0. Grace time: 2s.
Weighting	Final = Eye × 0.6 + Head × 0.4
Smoothing	3-second sliding window average
Calibration	First 3 seconds: learn user's normal head position as baseline
Status	>0.7 = Active, 0.4-0.7 = Semi-active, <0.4 = Not active
Visibility	Host-only. Participants cannot see any attention data. Host's own % excluded from overall meter.
Emit Rate	Score sent to server every 1 second

4.4 Voice Captions (English Only)

Feature	Description
Speech-to-Text	Browser SpeechRecognition API (English). Always runs when mic is on for transcript capture.
Subtitle Display	Toggle in Settings. Controls visibility only — transcript always captured for AI summary.
Mute Protection	STT stops when mic muted to prevent ghost captions from speaker audio.

5. Architecture



6. Socket.io Events

6.1 Client → Server

Event	Payload	Purpose
room:create	{ }	Host creates new meeting room
room:join	{ code }	Participant joins room
room:leave	—	Leave meeting gracefully
room:end	—	Host ends meeting for all
signal	{ targetId, signal }	WebRTC SDP/ICE signaling
chat:message	{ text }	Send chat message
caption:voice	{ text, isFinal }	English voice caption from STT
screen:request	—	Guest requests screen share
screen:respond	{ targetId, approved }	Host approves/denies
attention:status	{ status, score }	Attention score (0-1)

6.2 Server → Client

Event	Payload	Purpose
peer:joined	{ peerId, name }	New participant joined
peer:left	{ peerId }	Participant left
signal	{ fromId, signal }	Relay WebRTC signaling
chat:message	{ text, from, at, socketId }	Broadcast chat
caption:voice	{ text, from, at, socketId, isFinal }	Broadcast voice caption
attention:update	{ peerId, name, status, score }	To host only
room:ended	{ reason, summaryCode, attentionData }	Meeting ended
screen:approved / denied	—	Screen share response

doc:processing / ready / error	{ fileName, ... }	Document pipeline status
--------------------------------	-------------------	--------------------------

7. Server Library Modules

File	Purpose
lib/auth.js	JWT token signing/verification + better-auth session management
lib/rooms.js	In-memory room state: create, join, leave, end, participant metadata
lib/meetingTranscript.js	Collect voice and chat entries during meeting for AI summary
lib/meetingSummary.js	Generate AI meeting summary + Q&A via DeepSeek
lib/meetingHistory.js	Persist/query meetings in Supabase for all participants
lib/summarize.js	Document insight generation (overview, key points, etc.) via DeepSeek
lib/geminiDocChat.js	Document Q&A — RAG-style answers using extracted text
lib/geminiKeys.js	DeepSeek API key pool with rate-limit rotation
lib/documentExtract.js	Multi-format text extraction (PDF, Word, Excel, PPT, code, etc.)
lib/meetingDocumentPipeline.js	Upload → extract → analyze → emit to clients
lib/roomDocuments.js	Per-room document state (shared + per-user private docs)
lib/supabase.js	Supabase client initialization

8. Database Schema

8.1 SQLite (Local — User Auth)

Table	Columns
user	id, name, email, emailVerified, image, createdAt, updatedAt
session	id, expiresAt, token, ipAddress, userAgent, userId, createdAt, updatedAt
account	id, accountId, providerId, userId, accessToken, refreshToken, password, ...

8.2 Supabase PostgreSQL (Cloud — Meeting History)

Table	Columns
meetings	id (UUID), room_code, meeting_name, summary, topics (JSONB), assignments (JSONB), key_decisions (JSONB), participant_summaries (JSONB), attention_stats (JSONB), raw_transcript, duration_minutes, attendees (TEXT[]), host_email, created_at
meeting_participants	id (UUID), meeting_id (FK), user_email, user_name, is_host, created_at. UNIQUE(meeting_id, user_email)

9. Deployment

Component	Detail
Platform	Railway (Docker-based)
Dockerfile	node:20-slim + build tools (python3, make, g++) for better-sqlite3
Start Command	node server.js
Auto-deploy	On git push to connected GitHub repo
Environment Variables	DEEPSEEK_API_KEY, SUPABASE_URL, SUPABASE_ANON_KEY, BETTER_AUTH_SECRET, BETTER_AUTH_URL, JWT_SECRET

10. Security

Measure	Implementation
---------	----------------

Authentication	JWT tokens (7-day expiry) + better-auth cookie sessions
Socket Auth	JWT verified on every socket connection
Room Access	Participants must be in room to upload docs, chat, or access features
Rate Limiting	Voice caption flood protection (20/sec), doc upload cooldown (8s), doc Q&A cooldown (5s)
Input Sanitization	All user text truncated/sanitized before storage or broadcast
File Upload	20MB limit, UUID filenames, restricted to valid document types
Attention Privacy	Detection runs in-browser. Only numeric scores (0-1) sent to server. Data visible to host only.
Summary Privacy	Attention stats shown only to host in summary and export. Participants see summary without attention data.
CORS	Origin-based CORS with credential support